

COSC 360 Project – System & Implementation Description

1. Overview

The application follows a three-tier architecture:

React Frontend (Port 4000) —> Node/Express Backend (Port 3000) —> MongoDB (Port 27017)

All three tiers run in Docker containers orchestrated by docker-compose.yml.

2. Features Implemented

The following core features were implemented in the system:

- **authentication and user management:** user registration and login, JWT-based authentication, user data stored in MongoDB, role-based access (admin vs regular user)
- **Social OAuth Integration:** simplified login and registration via Google and Facebook using Passport.js, featuring automatic profile image synchronization and "Find-or-Create" account logic.
- **course functionality:** browse and search courses, view course details, join courses, course data stored in database
- **discussion system:** posts, replies, attachments, stored in MongoDB with asynchronous updates
- **admin features:** admin can delete any post, admin has elevated permissions compared to regular users
- **security and validation:** passwords are hashed before storage, input validation on both client and server, basic sanitization of user input

3. File Structure

Backend (backend/src/)

File/Folder	Description
app.js	Express app setup, CORS, JSON parsing, static file serving for uploads, route mounting
service.js	Entry point, starts the server and triggers database seeding
routes/authRoutes.js	POST /signup, POST /login, GET /me
routes/courseRoutes.js	Full CRUD for courses, members, posts, replies, attachments
routes/userRoutes.js	User search, profile update, enable/disable
controllers/authController.js	Signup (bcrypt hash), login (JWT issue), getCurrentUser
controllers/courseController.js	~18 handler functions for courses, discussions, members
controllers/userController.js	Profile update, user search (with post-text matching), toggle status
middleware/authMiddleware.js	JWT verification, attaches req.user or returns 401
middleware/uploadMiddleware.js	Multer config for profile image uploads
models/User.js	Schema: username, email, passwordHash, role, profileImage, bio, isDisabled
models/Course.js	Schema: title, description, type, field, tags, duration, positions, memberCount, createdBy
models/Post.js	Schema: text, attachments[], replies[], viewCount, downloadCount, course ref, author ref
models/CourseMember.js	Junction table: courseId + userId + roleInCourse; unique compound index

seed.js	Idempotent seeder, creates 2 test users + 10 default courses on first run
---------	---

Frontend (frontend/src/)

File/Folder	Purpose
main.jsx	React entry point
App.jsx	Manages global state (page, role, currentUser, selectedCourseId) and conditional page rendering. Acts as a simple client-side router.
pages/CoverPage.jsx	Landing/splash page
pages/Home.jsx	Course browsing, async search, filters, spotlight, favourites, profile edit overlay
pages/Login.jsx	Login form with JWT handling
pages/Signup.jsx	Registration form with optional profile image upload
pages/CourseOverview.jsx	Course details, join/leave, recent posts preview
pages/CourseDiscussion.jsx	Create posts, reply, delete, file attachments
pages/CourseTeam.jsx	Members list with admin removal capability
pages/Dashboard.jsx	User's joined courses with leave functionality
pages/CreateCourse.jsx	Course creation form with rich fields
pages/ProfileEdit.jsx	Profile edit overlay with image preview and smart contain/cover fit detection
components/Navbar.jsx	Navigation icons (home, dashboard, notifications); auto-polls user status every 5s; handles logout
components/AdminActions.jsx	Edit course, manage members, delete course, user search with enable/disable
components/SearchBar.jsx	Debounced async search input

4. System Architecture

The frontend communicates with the backend via REST APIs, and the backend communicates with MongoDB using Mongoose models:

1. Frontend (React): Handles UI and user interaction, Sends HTTP requests to backend APIs
2. Backend (Express): Handles routing and business logic, Performs authentication and validation, Interacts with the database
3. Database (MongoDB): Stores users, courses, and discussion data

5. How the System Works (High-Level Flow)

1. Authentication flow:
 - a. User submits login credentials → backend hashes and compares password with bcrypt → issues a signed JWT (7-day expiry) → frontend stores token and user object in localStorage.
 - b. Every protected API request attaches Authorization: Bearer <token> → authMiddleware.js verifies the JWT and injects req.user.
2. Course system:
 - a. Courses are stored in MongoDB. The CourseMember collection is a junction table linking users to courses, with a unique compound index preventing duplicate joins.
 - b. memberCount on the Course document is updated on every join/leave to avoid aggregation on the home listing.
3. Discussion system:
 - a. Posts belong to a Course and have an Author. Replies are embedded as a subdocument array inside the Post (denormalized for read efficiency).

- b. File attachments are stored via Multer to backend/uploads/ and served as static files at /uploads/<filename>. Attachment metadata (fileName, fileUrl, fileType, uploadedBy) is stored in the Post document.
 - c. Delete authorization is enforced server-side: regular users may only delete their own content (req.user.userId === post.author); admins may delete anything.
- 4. State management:
 - a. No Redux or external state library. Global state (current user, page, selected course) is managed in App.jsx and passed as props.

6. Database Design

The system uses MongoDB with the following collections that supports persistent storage of all user-generated content:

- Users: Stores user credentials and role (admin/user)
- Courses: Stores course information such as title, description, and tags
- Posts: Stores discussion threads, and includes replies and attachments

7. Deployment and Environment

The application is containerized using Docker, with the frontend, backend, and database defined in a single docker-compose.yml file. It can be started with one command: docker compose up --build. Services communicate through Docker networking, and MongoDB runs in a container, ensuring consistent and reproducible execution across environments.

8. Testing and Version Control

- The project uses Git for version control
- Core functionality was tested, including: Authentication, Course retrieval, Discussion posting and replying, API responses and error handling

9. Known Limitations

The current system has several limitations: No pagination for large datasets, File upload validation is basic, No real-time WebSocket implementation (uses polling / refresh logic), Limited UI optimization for mobile devices, Error messages could be more user-friendly

10. Future Improvements

Possible enhancements include:

- Implement real-time updates using WebSockets
- Improve UI/UX responsiveness
- Add notifications for new posts or replies
- Implement advanced search and filtering
- Add analytics or usage tracking features

11. Conclusion

This project demonstrates a complete full-stack web application with database integration, authentication, and interactive features. The system meets the core requirements of the project and provides a scalable foundation for further improvements.